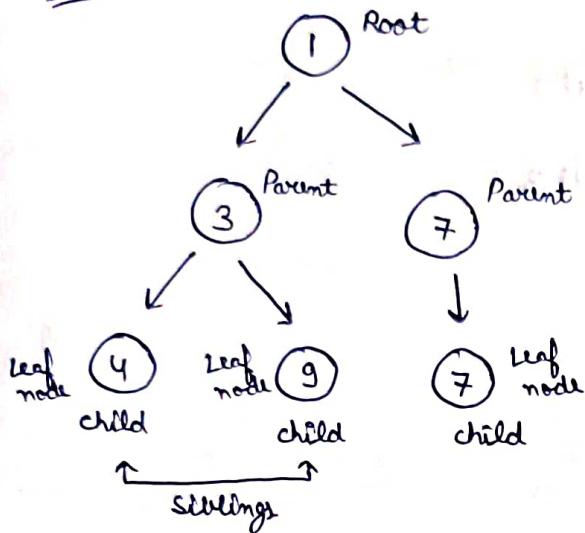


## Basic terms

- ① Node
- ② Root
- ③ Children
- ④ Parent
- ⑤ Siblings
- ⑥ Ancestor
- ⑦ Descendant
- ⑧ Leaf

## Tree:-



jo data contain karta hai

Node

int data;

Root → Jaha se tree start ho raha ho

⇒ 9 Jo hai wo 3 ka child hai to 3 parent ho gaya, similarly for 4

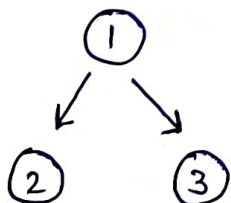
Siblings ⇒ Jiske parent same ho  
i.e. 9 & 4 are siblings

Ancestor ⇒ Koi ek node pakad lo aur upar jaste raho to wo ancestors honge & Descendant iska vice versa

Leaf ⇒ Jiska koi child nahi hai

⇒ Binary tree is that tree that has  $\leq 2$  child

## Example of Binary Tree:-



Node {

int data;

Node \* left;

Node \* right;

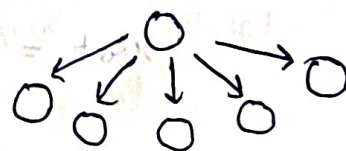
}

n-ary Tree (Multiple child)

Node {

int data;

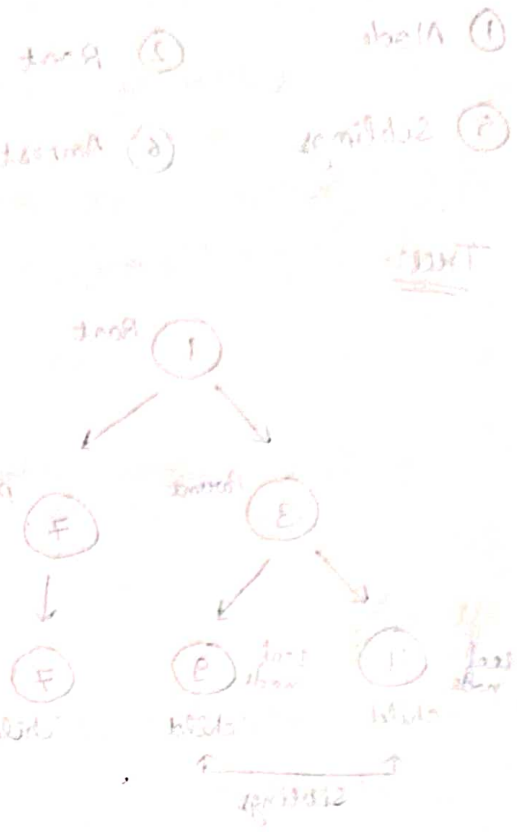
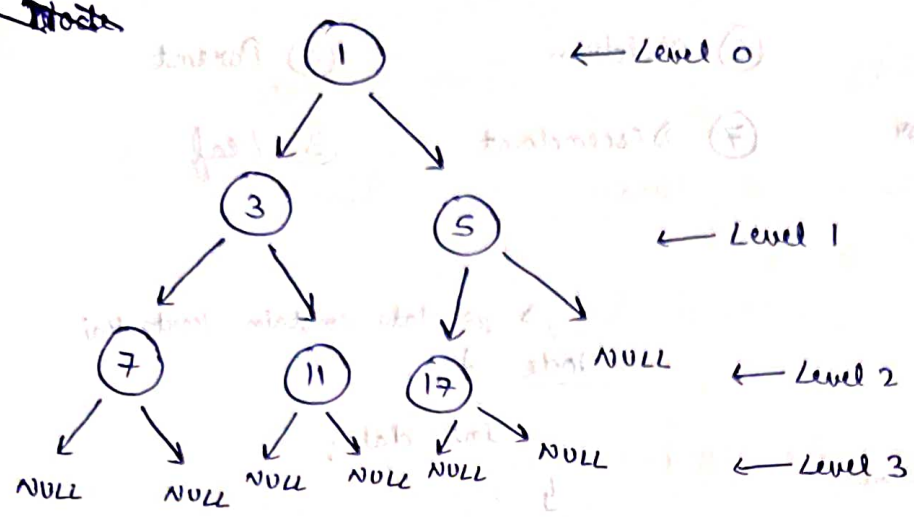
list < node \* > child;



# # Creation of Binary Tree

Insert 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

~~class Node~~



class node {

public:

int data;

node\* left;

node\* right;

node(int d) {

this->data = d;

this->left = NULL;

this->right = NULL;

}

};

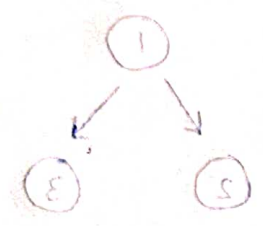
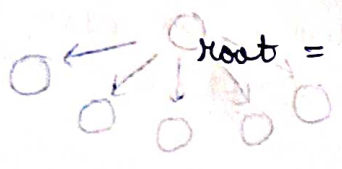
node\* buildTree(node\* root) {

cout << "Enter the data: ";

int data;

cin >> data;

root = new node(data);



(1) (3) (5) (7) (11) (17) (19) (21) (23) (25) (27) (29) (31) (33) (35) (37) (39) (41) (43) (45) (47) (49) (51) (53) (55) (57) (59) (61) (63) (65) (67) (69) (71) (73) (75) (77) (79) (81) (83) (85) (87) (89) (91) (93) (95) (97) (99)

```
if (data == -1) {
```

```
    return NULL;
```

```
    {
```

```
    cout << "Enter data for inserting in left of " << data << endl;
```

```
    root -> left = buildTree(root -> left);
```

```
    cout << "Enter data for inserting in right of " << data << endl;
```

```
    root -> right = buildTree(root -> right);
```

```
    return root;
```

```
}
```

Level order Traversal  $\Rightarrow$  BFS (Breadth First Search)

Level wise Tree print karna

$\Rightarrow$

```

    1          ← Level 0
   3  5       ← Level 1
  7 11 17     ← Level 2
  
```

```
void levelOrderTraversal (node* root) {
```

```
    queue < node* > q;
```

```
    q.push (root);
```

```
    q.push (NULL);
```

```
    while (!q.empty()) {
```

```
        node* temp = q.front();
```

```
        q.pop();
```

```
        if (temp == NULL) {
```

```
            return;
```

```
        }
```

```
        // print
```

```
        temp
```

```
        // print
```

```
        temp
```

```
        // print
```

```
        temp
```

```
        // print
```

```
        temp
```

```
        // print
```

```
        temp
```

```
        // print
```

```
        temp
```

```
        // print
```

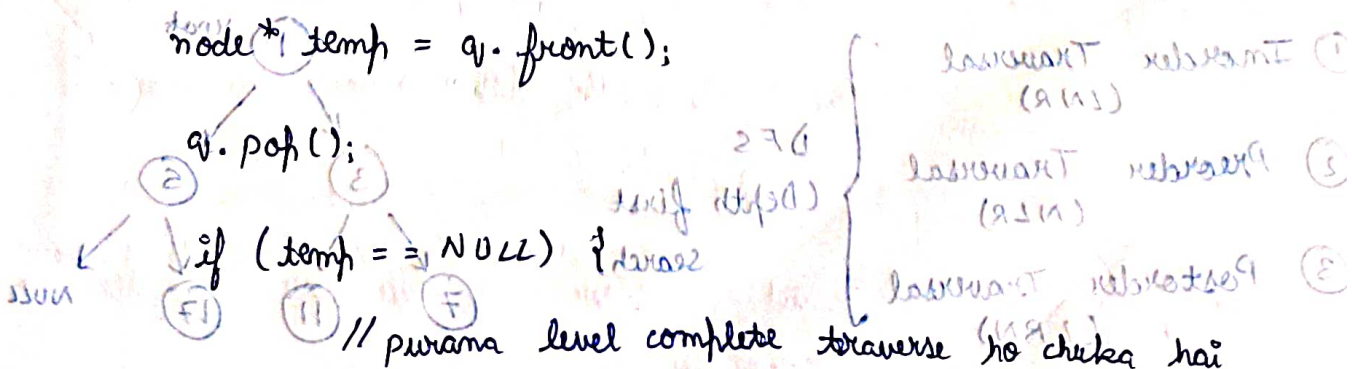
```
        temp
```

```
        // print
```

```
        temp
```

```
        // print
```

// print level complete traverse ho chuka hai





```
cout << endl;
```

```
if (!q.empty()) {
```

```
// queue still has some child nodes
```

```
q.push(NULL);
```

```
}
```

```
else {
```

```
cout << temp->data << " ";
```

```
if (temp->left) {
```

```
q.push(temp->left);
```

```
}
```

```
if (temp->right) {
```

```
q.push(temp->right);
```

```
}
```

```
}
```

```
}
```

```
}
```

H.W.

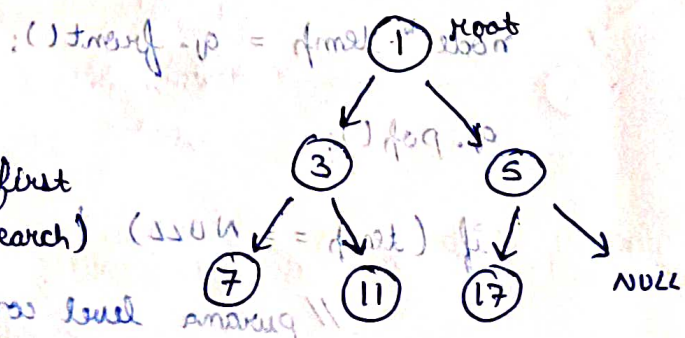
Code Reverse Level Order Traversal

① Inorder Traversal (LNR)

② Preorder Traversal (NLR)

③ Postorder Traversal (LRN)

DFS  
(Depth first search)





Inorder :- 7 3 11 1 17 5

(Left Node Right)

Preorder :- 1 3 7 11 5 17

(Node Left Right)

Postorder :- 7 11 3 17 5 1

(Left Right Node)

### Code for Inorder Traversal

```
void inorder (node * root) {
```

```
    // base case
```

```
    if (root == NULL) {
```

```
        return;
```

```
    }
```

```
    inorder (root -> left);
```

```
    cout << root -> data << " ";
```

```
    inorder (root -> right);
```

```
}
```

### Code for Preorder Traversal

```
void preorder (node * root) {
```

```
    if (root == NULL) {
```

```
        return;
```

```
    }
```

```
    cout << root -> data << " ";
```

```
    preorder (root -> left);
```

```
    preorder (root -> right);
```

```
}
```

T.C. =  $O(N)$

S.C. =  $O(N)$

## Code for Postorder Traversal

```
void postorder (node* root) {  
    if (root == NULL) {  
        return;  
    }  
    postorder (root->left);  
    postorder (root->right);  
    cout << root->data << " ";  
}
```

H.W.

Solve above three traversals with iteration (without using Recursion)

Qn- Create Tree from Level Order Traversal

```
void buildFromLevelOrder (node* &root) {
```

```
    queue <node*> q;
```

```
    cout << "Enter data for root" << endl;
```

```
    int data;
```

```
    cin >> data;
```

```
    root = new node (data);
```

```
    q.push (root);
```

```
    while (!q.empty()) {
```

```
        node* temp = q.front();
```

```
        q.pop();
```

```
        cout << "Enter left node for : " << temp->data << endl;  
        int leftdata;
```

```
cin >> leftData;
```

```
if (leftData != -1) {
```

```
    temp->left = new node(leftData);
```

```
    q.push(temp->left);
```

```
}
```

```
cout << "Enter right node for: " << temp->data << endl;
```

```
int rightData;
```

```
cin >> rightData;
```

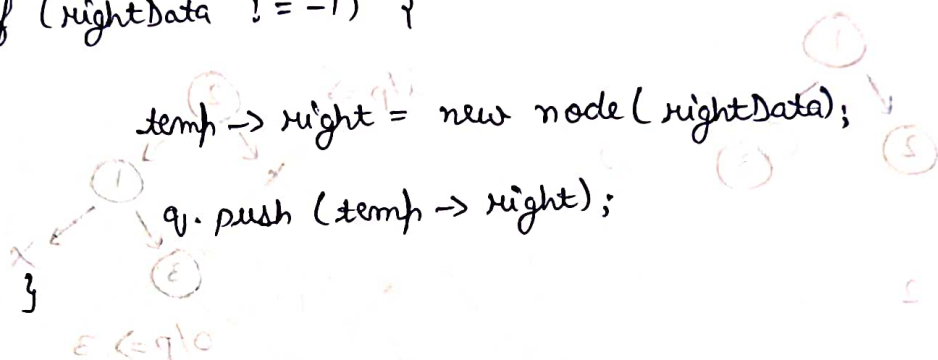
```
if (rightData != -1) {
```

```
    temp->right = new node(rightData);
```

```
    q.push(temp->right);
```

```
}
```

```
}
```



(If you want to see the full code, please refer to the link: <https://www.geogebra.org/m/1234567890>)

Qu- Count leaf nodes

```
int noOfLeafNodes (BinaryTreeNode <int> * root) {
```

```
    int cnt = 0;
```

```
    inorder (root, cnt);
```

```
    return cnt;
```

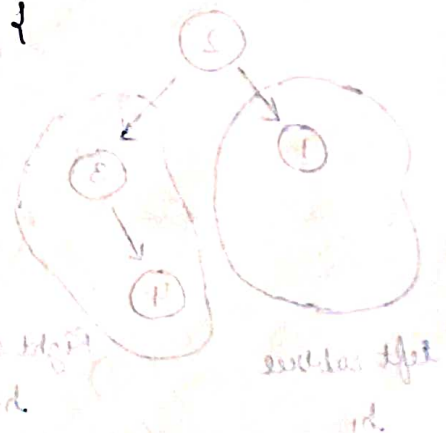
```
}
```

```
void inorder (BinaryTreeNode <int> * root, int & count) {
```

```
    if (root == NULL) {
```

```
        return;
```

```
}
```





```
inorder (root → left, count);
```

```
// leaf node
```

```
if (root → left == NULL && root → right == NULL) {
```

```
count++;
```

```
}
```

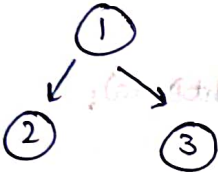
```
inorder (root → right, count);
```

```
}
```

### Lecture 63

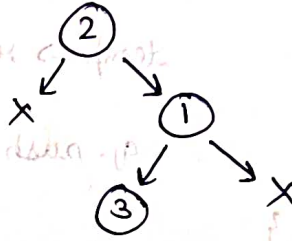
Ques- Height of Binary Tree

i/p ⇒



o/p ⇒ 2

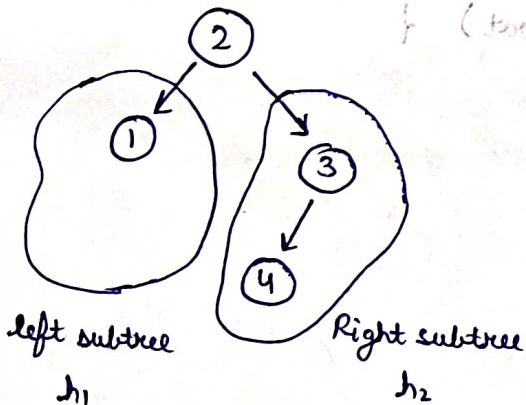
i/p ⇒



o/p ⇒ 3

Height ⇒ Longest path b/w root node & a leaf node (no of nodes = height)

### Approach



$\max.(h_1, h_2) + \text{root node} = \text{height}$   
(i.e., 1)

(By Recursion)

```
int height(struct Node* node) {
```

```
if (node == NULL) {
```

```
return 0;
```

3

```
int left = height(node->left);
```

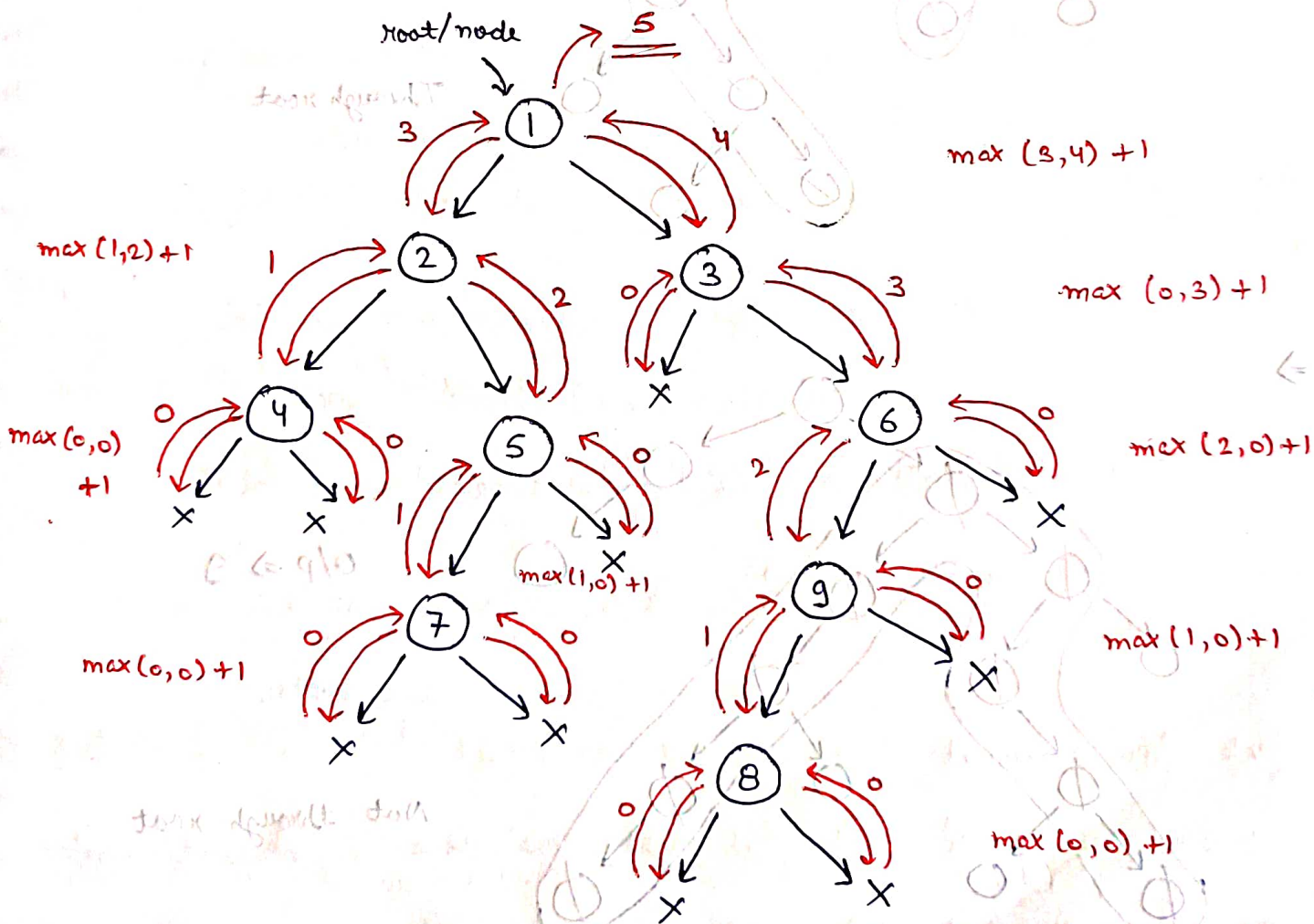
```
int right = height (node -> right);
```

```
int ans = max(left, right) + 1;
```

```
return ans;
```

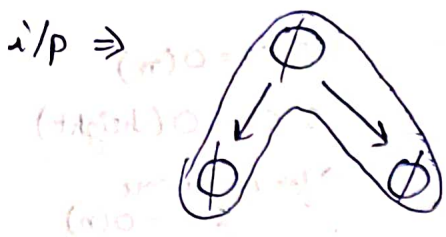
3

Dry Run:-



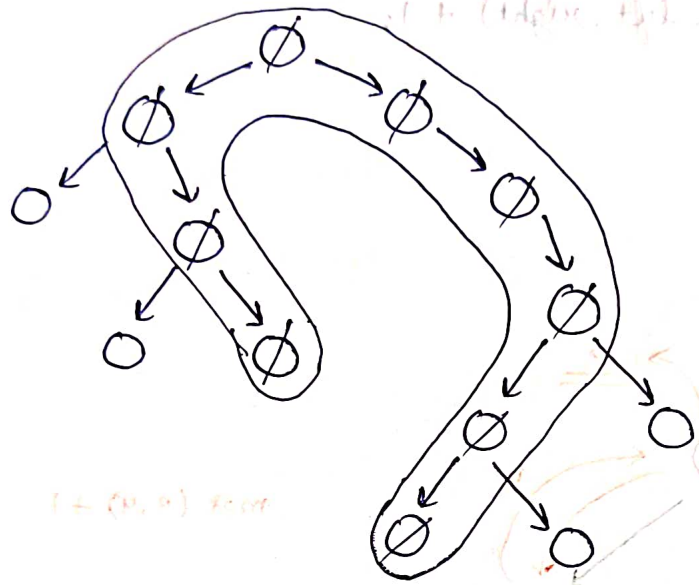
So, Height of tree = 5

Ques- Diameter of a Binary Tree (longest path b/w any two nodes i.e. no. of nodes = diameter)



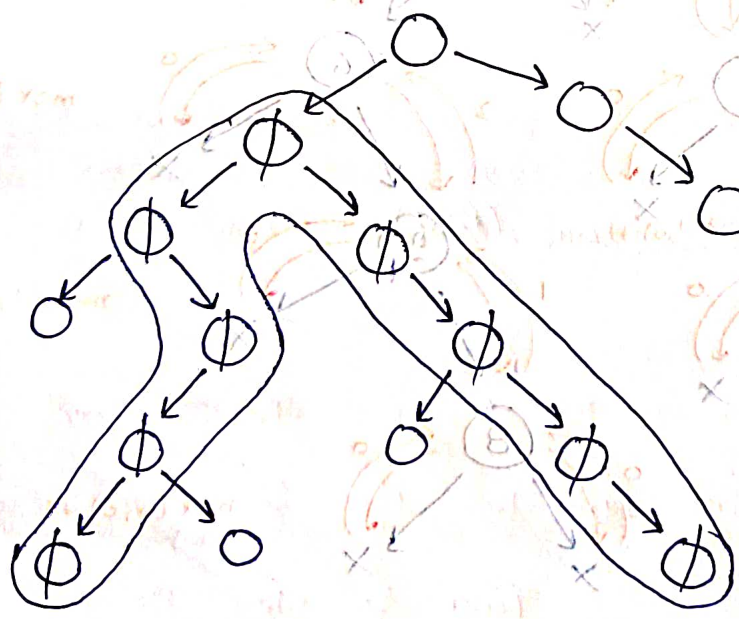
o/p  $\Rightarrow 3$

i/p  $\Rightarrow$



o/p  $\Rightarrow 9$

i/p  $\Rightarrow$



Through root

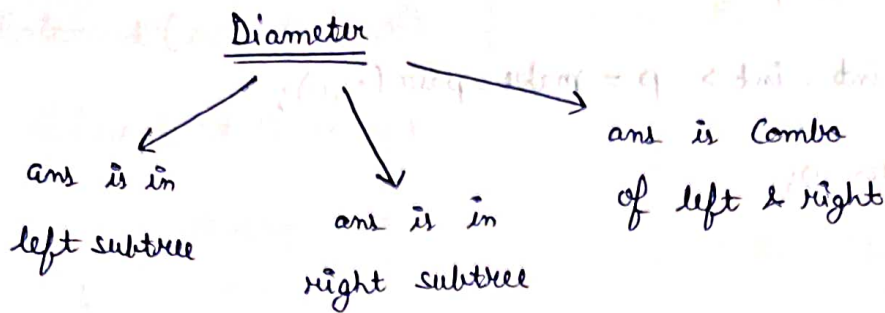
o/p  $\Rightarrow 9$

Not through root

3 = max of left, right



## Approach



option 1 = diameter (root  $\rightarrow$  left)

option 2 = diameter (root  $\rightarrow$  right)

option 3 = height (root  $\rightarrow$  left) + 1 + height (root  $\rightarrow$  right)

Now,  $\max(\text{option 1, option 2, option 3}) \Rightarrow \underline{\underline{\text{answer}}}$

```
int diameter(Node* root) {
```

```
    if (root == NULL) {
```

```
        return 0;
```

```
    }
```

T.C =  $O(n^2)$

```
    int op1 = diameter(root  $\rightarrow$  left);
```

```
    int op2 = diameter(root  $\rightarrow$  right);
```

```
    int op3 = height(root  $\rightarrow$  left) + height(root  $\rightarrow$  right) + 1;
```

```
    int ans = max(op1, op2, op3);
```

```
    return ans;
```

height function is same as previous

But there is a problem in this sol<sup>n</sup> as the T.C becomes higher

because we are calling height fn inside this (T.C =  $O(n^2)$ )

pair <int, int> diameterFast (Node\* root) {

if (root == NULL) {

pair <int, int> p = make\_pair(0, 0);

return p;

}

pair <int, int> left = diameterFast (root -> left);

pair <int, int> right = diameterFast (root -> right);

int op1 = left.first;

int op2 = right.first;

int op3 = left.second + right.second + 1;

pair <int, int> ans;

ans.first = max(op1, max(op2, op3));

ans.second = max(left.second, right.second) + 1;

return ans;

}

int diameter (Node\* root) {

return diameterFast (root).first;

}

T.C. =  $O(n)$

S.C. =  $O(n)$

Qn- Check for Balanced Tree (A tree is height balanced if difference between heights of left and right subtrees is not more than one for all nodes of tree)

$$\text{i.e. } |(\text{Height})_{\text{left}} - (\text{Height})_{\text{right}}| \leq 1$$

### Approach ①

```
bool isBalanced (Node* root) {
```

```
    if (root == NULL) {
```

```
        return true;
```

```
    }
```

```
    bool left = isBalanced (root -> left);
```

```
    bool right = isBalanced (root -> right);
```

```
    bool diff = abs (height (root -> left) - height (root -> right)) <= 1;
```

```
    if (left && right && diff) {
```

```
        return true;
```

```
    }
```

```
    else {
```

```
        return false;
```

```
    }
```

```
}
```

T.C. =  $O(n^2)$

↳ Because height fn  
is also called

### Approach ②

```
pair < bool, int > isBalancedFast (Node* root) {
```

```
    if (root == NULL) {
```

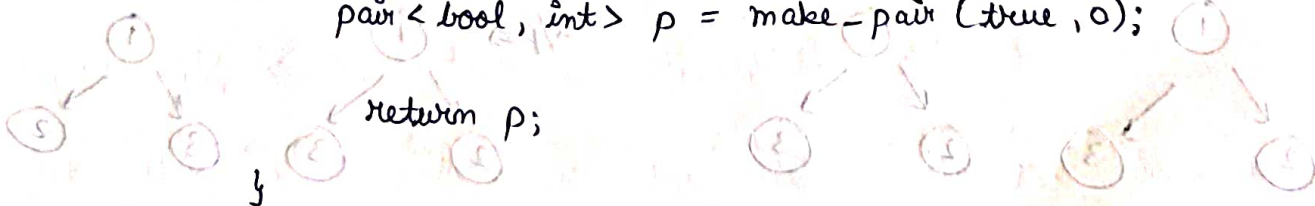
```
        pair < bool, int > p = make_pair (true, 0);
```

```
        return p;
```

```
    }
```

```
    pair < bool, int > left = isBalancedFast (root -> left);
```

```
    pair < bool, int > right = isBalancedFast (root -> right);
```





bool leftAns = left.first;

bool rightAns = right.first;

bool diff = abs(left.second - right.second) <= 1;

pair < bool, int > ans;

ans.second = max(left.second, right.second) + 1;

if (leftAns & rightAns & diff) {

ans.first = true;

}

else {

ans.first = false;

}

return ans;

}

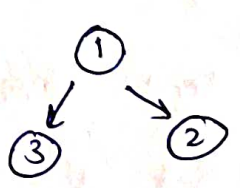
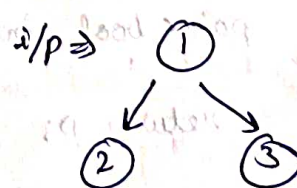
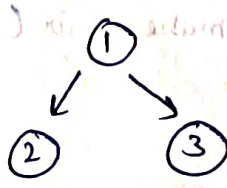
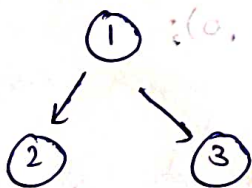
bool isBalanced(Node\* root) {

return isBalancedFast(root).first;

}

Qn- Determine if two trees are identical

i/p =>



o/p => Yes

o/p => No

bool isIdentical (Node \*H1, Node \*H2) {

if (H1 == NULL && H2 == NULL) {

return true;

if (H1 == NULL && H2 != NULL) {

return false;

if (H1 != NULL && H2 == NULL) {

return false;

bool left = isIdentical (H1->left, H2->left);

bool right = isIdentical (H1->right, H2->right);

bool value = H1->data == H2->data;

if (left && right && value) {

return true;

else {

return false;

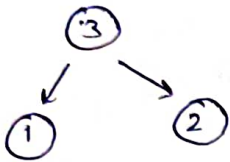
T.C. = O(m)

H.D.

Draw Recursive tree of prev. question

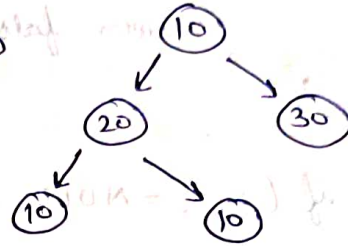
Ques- Given a Binary Tree, Return true if for every node  $x$  in the tree other than the leaves, its value is equal to the sum of its left subtree's value and its right subtree's value. Else return false. An empty tree is also a sum tree as the sum of an empty tree can be considered to be 0. A leaf node is also considered a sum tree.

i/p  $\Rightarrow$



o/p  $\Rightarrow$  1

i/p  $\Rightarrow$



o/p  $\Rightarrow$  0

```

pair < bool, int > isSumTreeFast (Node * root) {
    if (root == NULL) {
        pair < bool, int > p = make_pair (true, 0);
        return p;
    }
    if (root -> left == NULL && root -> right == NULL) {
        pair < bool, int > p = make_pair (true, root -> data);
        return p;
    }
    pair < bool, int > leftAns = isSumTreeFast (root -> left);
    pair < bool, int > rightAns = isSumTreeFast (root -> right);
    bool left = leftAns.first;
    bool right = rightAns.first;
  
```



bool condn = root->data == leftms.second + rightms.second; done

pair < bool, int > ans;

if (left && right && condn) {

ans.first = true;

ans.second = 2 \* root->data;

}

else {

ans.first = false;

}

return ans;

}

bool isSumTree (Node\* root) {

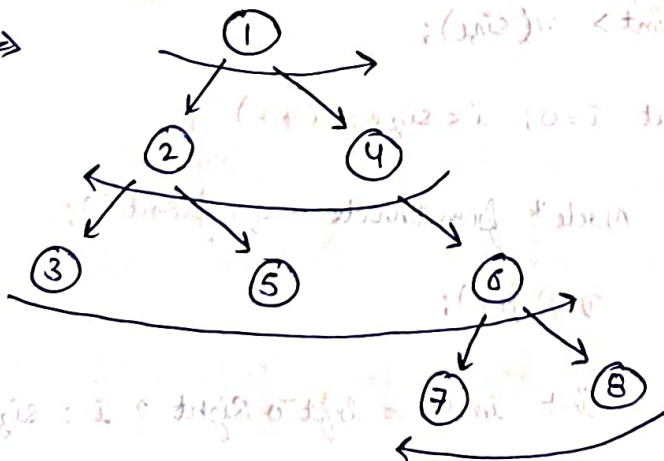
return isSumTree<sup>Fast</sup>(root).first;

}

## Lecture 64

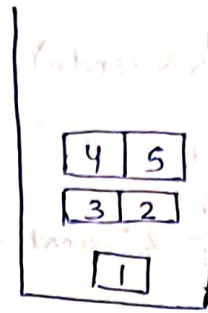
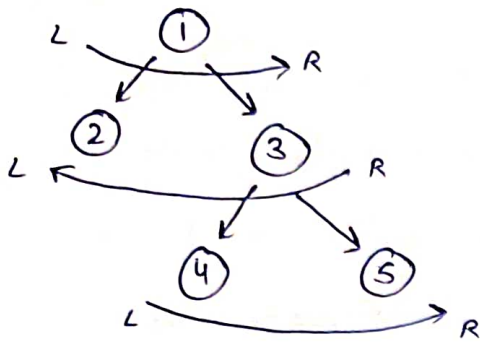
### Qu-Zig-Zag Traversal (Spiral Traversal)

l/p ⇒



O/p ⇒ 1 4 2 3 5 6 8 7

## Approach



pop & store in ans

O/P  $\Rightarrow$  1 3 2 4 5

flag

left to right = True  
right to left = False

```
vector<int> zigZagTraversal(Node* root) {
```

```
    vector<int> ans;
```

```
    if (root == NULL) {
```

```
        return ans;
```

```
    }
```

```
    queue<Node*> q;
```

```
    q.push(root);
```

```
    bool leftToRight = true;
```

```
    while (!q.empty()) {
```

```
        int size = q.size();
```

```
        vector<int> v(size);
```

```
        for (int i = 0; i < size; i++) {
```

```
            Node* frontNode = q.front();
```

```
            q.pop();
```

```
            int index = leftToRight ? i : size - i - 1;
```

```
            v[index] = frontNode->data;
```

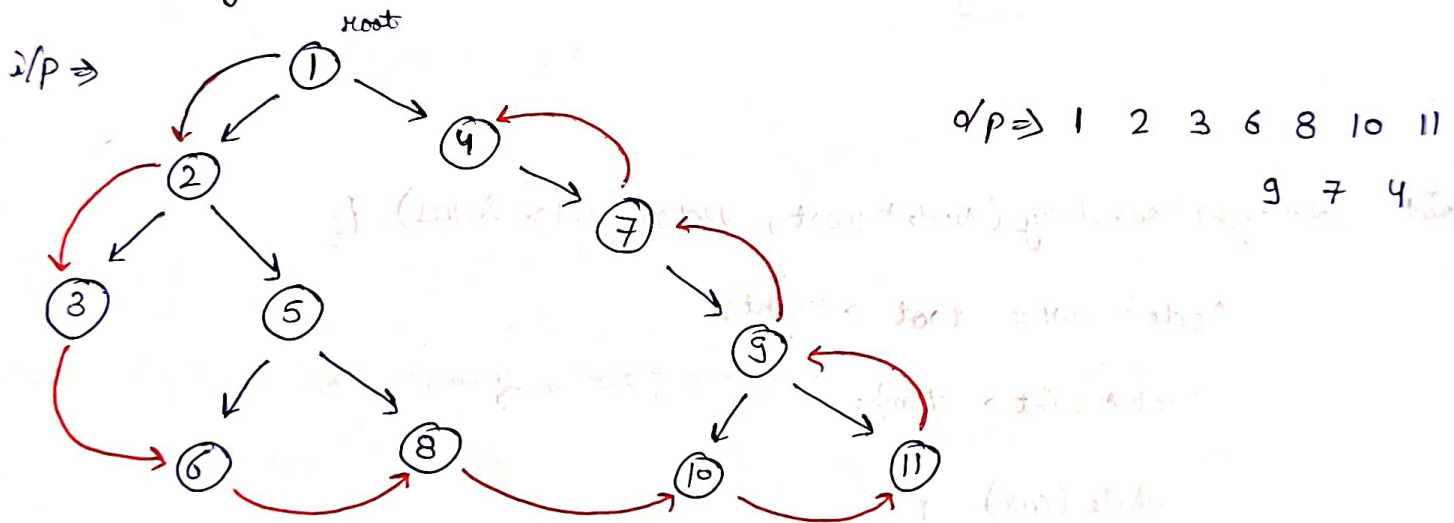
```

if (frontNode -> left) {
    q.push(frontNode -> left);
}
if (frontNode -> right) {
    q.push(frontNode -> right);
}
}
leftToRight = !leftToRight;
for (auto i : ans) {
    ans.push_back(i);
}
}
return ans;
}

```

T.C =  $O(n)$   
S.C =  $O(n)$

### Ques- Boundary Traversal



### Algo:-

- ① print left part except leaf node (1 2)
- ② print leaf nodes (L  $\rightarrow$  R) { via Inorder Traversal }
- ③ print right part except leaf node in reverse order (stack can be used) for reversing



```
void addLeftBoundary (Node* root, vector<int> &res) {
```

```
    Node* cur = root->left;
```

```
    while (cur) {
```

```
        if (!isLeaf(cur)) {
```

```
            res.push_back(cur->data);
```

```
        }
```

```
        if (cur->left) {
```

```
            cur = cur->left;
```

```
        }
```

```
    } else {
```

```
        cur = cur->right;
```

```
    }
```

```
}
```

```
}
```

```
void addRightBoundary (Node* root, vector<int> &res) {
```

```
    Node* cur = root->right;
```

```
    vector<int> temp;
```

```
    while (cur) {
```

```
        if (!isLeaf(cur)) {
```

```
            temp.push_back(cur->data);
```

```
        }
```

```
        if (cur->right) {
```

```
            cur = cur->right;
```

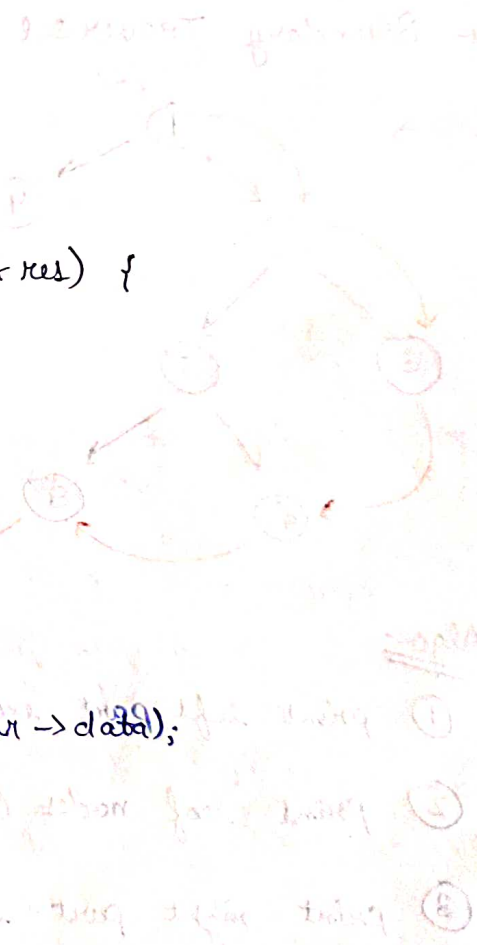
```
        }
```

```
    } else {
```

```
        cur = cur->left;
```

```
    }
```

```
}
```



```
for (int i = temp.size() - 1; i >= 0; i--) {
```

```
    res.push_back(temp[i]);
```

```
}
```

```
}
```

```
void addLeaves (Node * root, vector<int> & res) {
```

```
    if (isLeaf (root)) {
```

```
        res.push_back (root->data);
```

```
        return;
```

```
    }
```

```
    if (root->left) {
```

```
        addLeaves (root->left, res);
```

```
    }
```

```
    if (root->right) {
```

```
        addLeaves (root->right, res);
```

```
    }
```

```
}
```

```
vector<int> printBoundary (Node * root) {
```

```
    vector<int> res;
```

```
    if (!root) {
```

```
        return res;
```

```
    }
```

```
    if (!isLeaf (root)) {
```

```
        res.push_back (root->data);
```

```
    }
```

```
    addLeftBoundary (root, res);
```

```
    addLeaves (root, res);
```

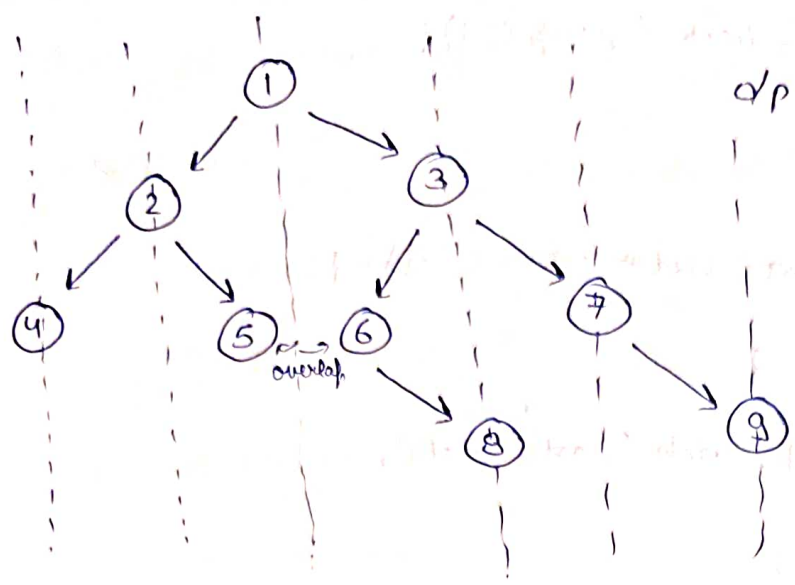
```
    addRightBoundary (root, res);
```

```
    return res;
```

```
}
```

# Qn- Vertical Order Traversal

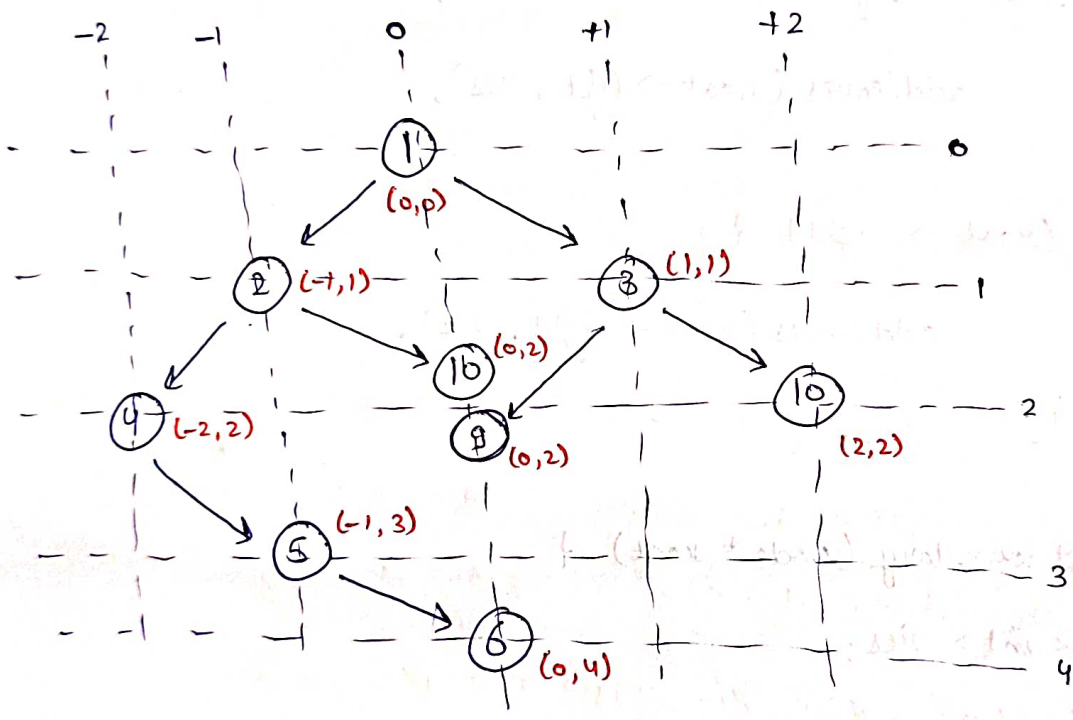
i/p =>



o/p => 4 2 1 5 6 3 8 7 9

There may be multiple nodes in the same row and same column. In such a case sort these nodes by their values.

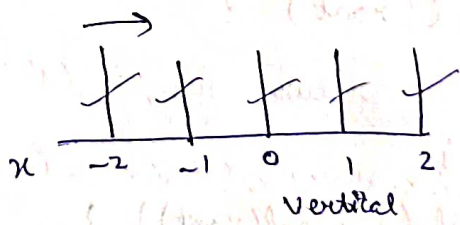
## Approach



o/p =>

4  
2 5  
1 9 10 6  
3  
10

x →  
↓ level



| node      | left      | right     |
|-----------|-----------|-----------|
| (1, 0, 0) | (2, 1, 1) | (3, 1, 1) |

q (node, vertical, level)

0 → 0 → {1}  
-1 → 1 → {2}

map<int, map<int, multiset<int>>>  
vertical level multi nodes



Vector <vector<int>> verticalTraversal(TreeNode\* root) {

map<int, map<int, multiset<int>>> nodes;

queue<pair<TreeNode\*, pair<int, int>>> todo;

todo.push({root, {0,0}});

while(!todo.empty()) {

auto p = todo.front();

todo.pop();

TreeNode\* node = p.first;

int x = p.second.first;

int y = p.second.second;

nodes[x][y].insert(node->val);

if (node->left) {

todo.push({node->left, {x-1, y+1}});

if (node->right) {

todo.push({node->right, {x+1, y+1}});

}

vector<vector<int>> ans;

for (auto p : nodes) {

vector<int> col;

for (auto q : p.second) {

col.insert(col.end(), q.second.begin(), q.second.end());

ans.push\_back(col);

return ans;

2<sup>nd</sup> code

```
vector<int> verticalOrder(Node * root) {
```

```
    map<int, map<int, vector<int>>> nodes;
```

```
    queue<pair<Node*, pair<int, int>>> q;
```

```
    vector<int> ans;
```

```
    if (root == NULL) {
```

```
        return ans;
```

```
    }
```

```
    q.push(make_pair(root, make_pair(0,0)));
```

```
    while (!q.empty()) {
```

```
        pair<Node*, pair<int, int>> temp = q.front();
```

```
        q.pop();
```

```
        Node* frontNode = temp.first;
```

```
        int hd = temp.second.first;
```

```
        int lvl = temp.second.second;
```

```
        nodes[hd][lvl].push_back(frontNode->data);
```

```
        if (frontNode->left) {
```

```
            q.push(make_pair(frontNode->left, make_pair(hd-1, lvl+1)));
```

```
        }
```

```
        if (frontNode->right) {
```

```
            q.push(make_pair(frontNode->right, make_pair(hd+1, lvl+1)));
```

```
        }
```

```
    }
```

```
for (auto i : nodes) {
```

```
    for (auto j : i.second) {
```

```
        for (auto k : j.second) {
```

```
            ans.push-back(k);
```

```
        }
```

```
    }
```

```
}
```

```
return ans;
```

```
}
```

Qn- Top View of Binary Tree

i/p  $\Rightarrow$

o/p  $\Rightarrow$  4 2 1 3 7

$\langle 0 \rangle \rightarrow 1$

$\langle -1 \rangle \rightarrow 2$

$\langle 1 \rangle \rightarrow 3$

$\langle -2 \rangle \rightarrow 4$

$\langle 2 \rangle \rightarrow 7$

1 to 1 mapping of horizontal dist. & node

map  $\langle \underline{\text{int}}, \underline{\text{int}} \rangle$

hd node  $\rightarrow$  data

```
Vector<int> topView(Node* root) {
```

```
    Vector<int> ans;
```

```
    if (root == NULL) {
```

```
        return ans;
```

```
    }
```

```
    map<int, int> topNode;
```



```
queue < pair < Node*, int >> q;
```

```
q.push(make_pair(root, 0));
```

```
while(!q.empty()) {
```

```
    pair < Node*, int > temp = q.front();
```

```
    q.pop();
```

```
    Node* frontNode = temp.first;
```

```
    int hd = temp.second;
```

```
    if (topNode.find(hd) == topNode.end()) {
```

```
        topNode[hd] = frontNode->data;
```

```
    }
```

```
    if (frontNode->left) {
```

```
        q.push(make_pair(frontNode->left, hd-1));
```

```
    }
```

```
    if (frontNode->right) {
```

```
        q.push(make_pair(frontNode->right, hd+1));
```

```
    }
```

```
}
```

```
for (auto i : topNode) {
```

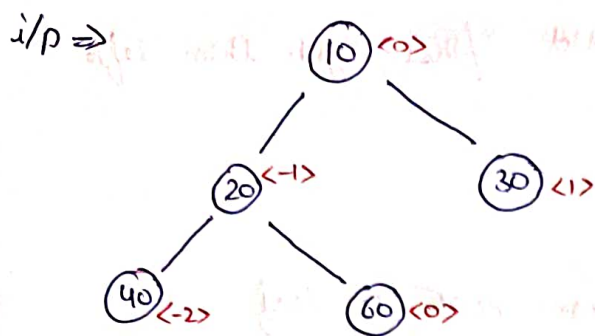
```
    ans.push_back(i.second);
```

```
}
```

```
return ans;
```

```
}
```

## Qn- Bottom View of Binary Tree



d/p  $\Rightarrow$  40 20 60 30

<0>  $\rightarrow$  10 60

<-1>  $\rightarrow$  20

<1>  $\rightarrow$  30

<-2>  $\rightarrow$  40

Code will be same as top view only have to

remove condition to check topNode.find(hel)

## Qn- Left View of Binary Tree

(Max level ki first node ko print karana hai)

void solve (Node\* root, vector<int> &ans, int level) {

if (root == NULL) {

return;

}

if (level == ans.size()) {

ans.push\_back (root->data);

}

solve (root->left, ans, level+1);

solve (root->right, ans, level+1);

}

vector<int> leftView (Node\* root) {

vector<int> ans;

solve (root, ans, 0);

return ans;

}

Q- Right View of Binary Tree

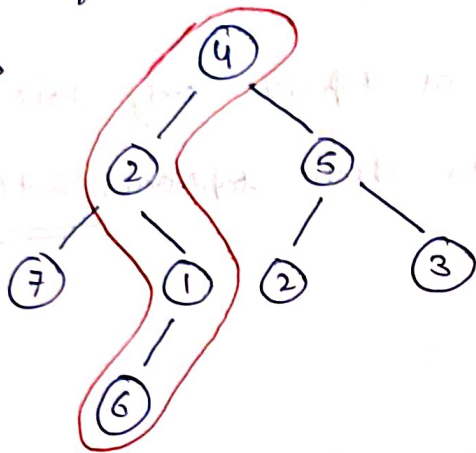
Now the call for the function is reversed first right then left

H.W.

Diagonal Traversal of Binary Tree

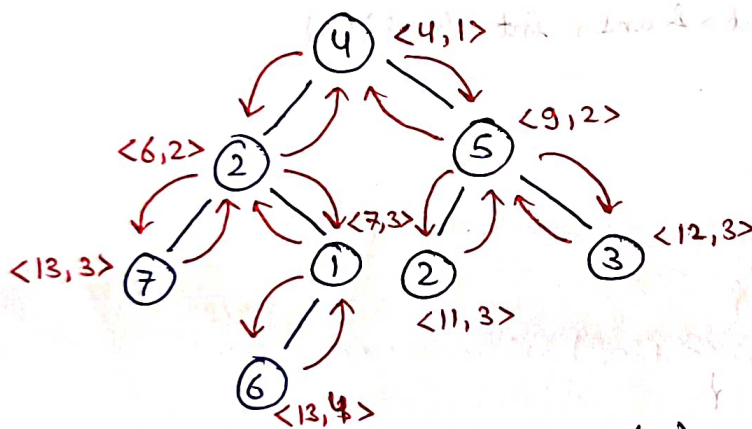
Q- Sum of nodes on the longest path from root to leaf

i/p  $\Rightarrow$



o/p  $\Rightarrow (4 + 2 + 1 + 6) = 13$

Approach



<sum, no. of nodes/length of path>

ans = <0,0>  $\rightarrow$   $\max(13,0)$

if (length  $\geq$ )  $\rightarrow$  & & > 0

ans = <13,3>  $\rightarrow$  4  $\geq$  3

ans = <max(13,13)>  $\Rightarrow$  <13,4>

3  $\geq$  4 False

3  $\geq$  4 False

return ans = <13,4>

T.C. =  $O(n)$

S.C. =  $O(H) \approx O(n)$

(Skewed Tree)

```
void solve(Node* root, int sum, int &maxsum, int len, int &maxlen) {
```

```
    if (root == NULL) {
```

```
        if (len > maxlen) {
```

```
            maxlen = len;
```

```
            maxsum = sum;
```

```
        }
```

```
else if (len == maxLen) {
```

```
    maxSum = max(sum, maxSum);
```

```
}
```

```
    return;
```

```
}
```

```
sum = sum + root->data;
```

```
solve(root->left, sum, maxSum, len+1, maxLen);
```

```
solve(root->right, sum, maxSum, len+1, maxLen);
```

```
}
```

```
int sumOfLongRootToLeafPath(Node* root) {
```

```
    int len = 0;
```

```
    int maxLen = 0;
```

```
    int sum = 0;
```

```
    int maxSum = INT_MIN;
```

```
    solve(root, sum, maxSum, len, maxLen);
```

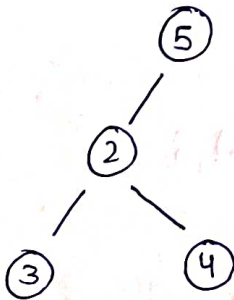
```
    return maxSum;
```

```
}
```

Ques - Lowest Common Ancestor in a Binary Tree

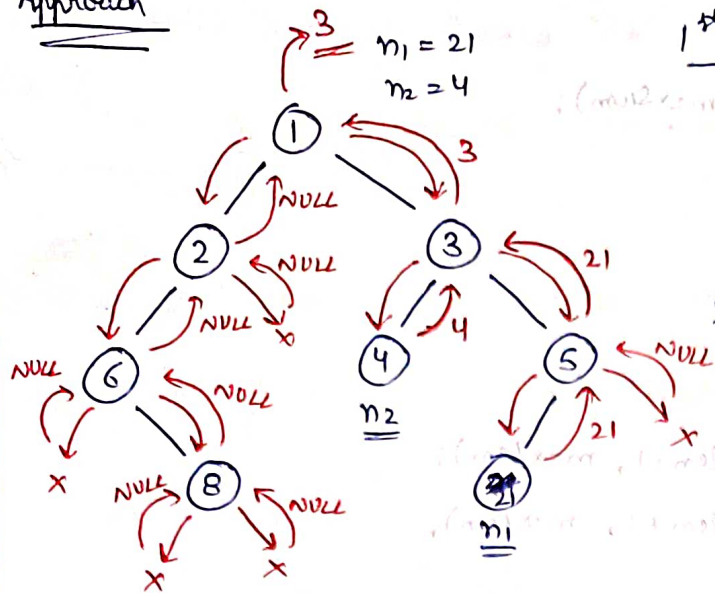
i/p  $\Rightarrow n_1 = 3, n_2 = 4$

o/p  $\Rightarrow 2$





### Approach



1<sup>st</sup> approach

Path of root  $\rightarrow 2n \Rightarrow 1 \mid 3 \mid 5 \mid 21$

Path of root  $\rightarrow n_2 \Rightarrow 1 \boxed{3} 4$   
LCA

2<sup>nd</sup> approach

left Ans & right Ans recursively

T.C. =  $O(n)$

$$S.C. = O(1) = \underline{O(1)}$$

## skew Tree

```
Node* lca(Node* root, int n1, int n2) { }
```

```
if (root == NULL) {
```

```
return NULL;
```

٢

```
if (root->data == n1 || root->data == n2) {
```

```
return root;
```

3

```
Node* leftAns = lca(root->left, n1, n2);
```

Node\* rightAns = lca (root->right, n1, n2);

```
if (leftAns != NULL &&
    rightAns != NULL)
    return root;
```

```

else if (leftAns != NULL && rightAns == NULL) {

```

return leftAns;

3

```
else if (leftAns == NULL && rightAns != NULL) {
```

```
return rightAns;
```

3

```
else {
```

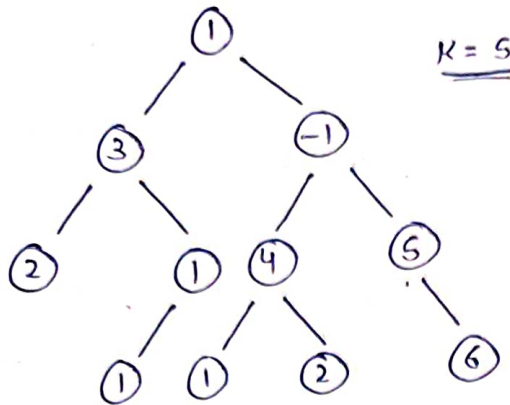
```
return NULL;
```

4

}

Q- K sum paths (Given a binary tree and an integer K. Find the no. of paths which have their sum equal to K)

i/p  $\Rightarrow$



o/p  $\Rightarrow$  8

Paths  $\Rightarrow$

```

3 2
3 1 1
1 3 1
4 1
1 -1 4 1
-1 4 2
5
1 -1 5
  
```

```

void solve (Node* root, int K, int & count, vector<int> path) {
  if (root == NULL) {
    return ;
  }
  path.push-back (root->data);
  solve (root->left, K, count, path);
  solve (root->right, K, count, path);

  int size = path.size();
  int sum = 0;
  for (int i = size-1; i >= 0; i--) {
    sum += path[i];
    if (sum == K) {
      count++;
    }
  }
  path.pop-back();
}
  
```

if (root == NULL) {

return ;

}

path.push-back (root->data);

solve (root->left, K, count, path);

solve (root->right, K, count, path);

int size = path.size();

int sum = 0;

for (int i = size-1; i >= 0; i--) {

sum += path[i];

if (sum == K) {

count++;

}

}

path.pop-back();

}

int sumK (Node\* root, int K) {

vector<int> path;

int count = 0;

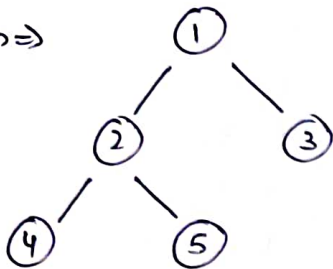
```
solve (root, K, count, path);
```

```
return count;
```

```
}
```

Qu- K<sup>th</sup> Ancestor

i/p  $\Rightarrow$



K = 2

node = 4

o/p  $\Rightarrow$  1

```
Node* solve (Node* root, int k, int node) {
```

```
    if (root == NULL) {
```

```
        return NULL;
```

```
    }
```

```
    if (root->data == node) {
```

```
        return root;
```

```
    }
```

```
    Node* leftAns = solve (root->left, k, node);
```

```
    Node* rightAns = solve (root->right, k, node);
```

```
    if (leftAns != NULL && rightAns == NULL) {
```

```
        k--;
```

```
        if (k <= 0) {
```

```
            k = INT_MAX;
```

```
            return root;
```

```
        }
```

```
        return leftAns;
```

```
    }
```

```
    if (leftAns == NULL && rightAns != NULL) {
```

```
        k--;
```

```
        if (k <= 0) {
```

$K = INT\_MAX;$

return root;

}

return rightAns;

}

return NULL;

}

int KthAncestor (Node \* root, int K, int node) {

Node \* ans = solve (root, K, node);

if (ans == NULL) { // ans -> data == node

return -1;

}

else {

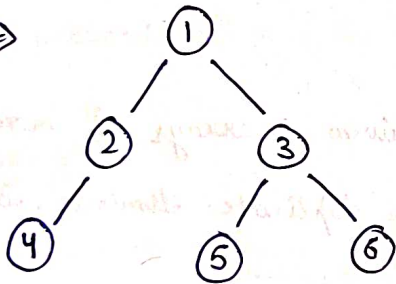
return ans -> data;

}

}

Q. Maximum sum of non-adjacent nodes

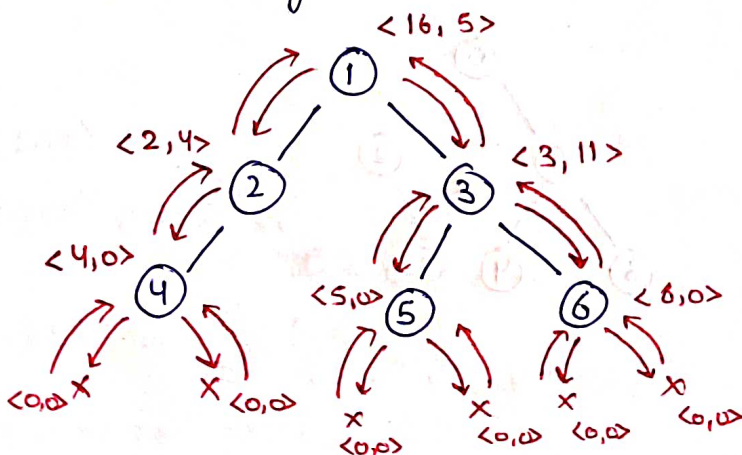
i/p  $\Rightarrow$



o/p  $\Rightarrow 6 + 5 + 1 + 4 = \underline{\underline{16}}$

Approach -

< max sum by inc. nodes at curr level, max sum by ~~inc~~ <sup>exc</sup> nodes at curr level:





```
pair<int, int> solve(Node* root) {
```

```
    if (root == NULL) {
```

```
        pair<int, int> p = make_pair(0, 0);
```

```
        return p;
```

```
    }
```

```
    pair<int, int> left = solve(root->left);
```

```
    pair<int, int> right = solve(root->right);
```

```
    pair<int, int> res;
```

```
    res.first = root->data + left.second + right.second;
```

```
    res.second = max(left.first, left.second) + max(right.first, right.second);
```

```
    return res;
```

```
}
```

```
int getMaxSum(Node* root) {
```

```
    pair<int, int> ans = solve(root);
```

```
    return max(ans.first, ans.second);
```

```
}
```

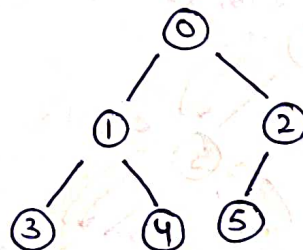
Qn- Construct ~~tree~~ from Inorder & Preorder (Given 2 arrays of inorder and preorder traversal. The tree can contain duplicate elements. Construct a tree and print the postorder traversal)

i/p  $\Rightarrow N=6$

inorder  $[\ ] = \{3, 1, 4, 0, 5, 2\}$

preorder  $[\ ] = \{0, 1, 3, 4, 2, 5\}$

o/p  $\Rightarrow \{3, 4, 1, 5, 2, 0\}$



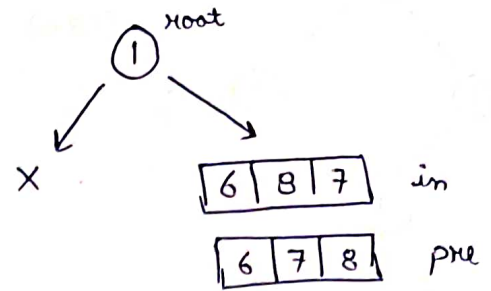
## Approach:

(LNR) in  $\Rightarrow$  [1 6 8 7]

(NLR) pre  $\Rightarrow$  [1 6 7 8]

(LNR) in  $\Rightarrow$  [6 8 7]

(NLR) pre  $\Rightarrow$  [6 7 8]

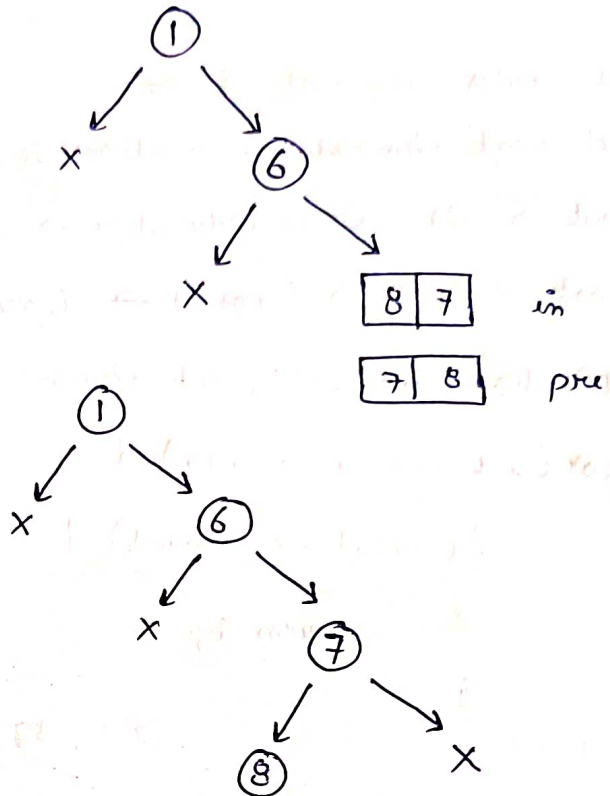


(LNR) in  $\Rightarrow$  [8 7]

(NLR) pre  $\Rightarrow$  [7 8]

(LNR) in  $\Rightarrow$  [8]

(NLR) pre  $\Rightarrow$  [8]

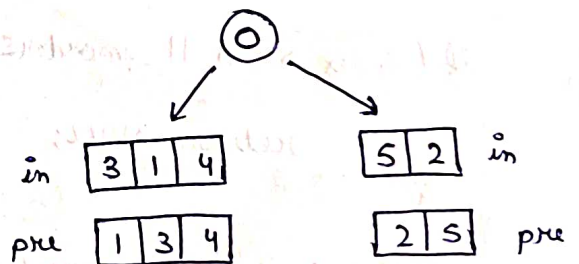


(LRN) postorder  $\Rightarrow$  [8 7 6 1]

## 2nd example

(LNR) in  $\Rightarrow$  [3 1 4 0 5 2]

(NLR) pre  $\Rightarrow$  [0 1 3 4 2 5]

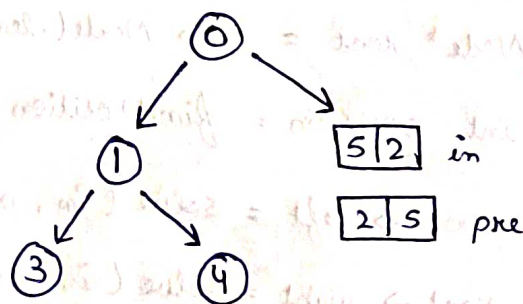


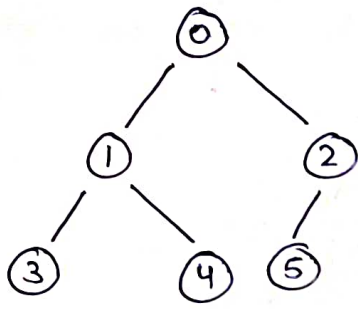
(LNR) in  $\Rightarrow$  [3 1 4]

(NLR) pre  $\Rightarrow$  [1 3 4]

(LNR) in  $\Rightarrow$  [5 2]

(NLR) pre  $\Rightarrow$  [2 5]





(LRN) postorder[] = [3 4 1 5 2 0]

Algo:-

- Take index as root element
- Find root element Ki position in inorder
- root → left ⇒ (inorder start → pos-1)
- root → right ⇒ (pos+1 → inorder end)

```

int findPosition (int in[], int element, int n) {
    for (int i=0; i<n; i++) {
        if (in[i] == element) {
            return i;
        }
    }
    return -1;
}

```

We can eliminate this by creating mapping of node to index of inorder at the start

```

Node* solve (int in[], int pre[], int & index, int inorderStart, int inorderEnd, int n) {
    if (index >= n || inorderStart > inorderEnd) {
        return NULL;
    }
    int element = pre[index++];
    Node* root = new Node(element);
    int position = findPosition (in, element, n);
    root->left = solve (in, pre, index, inorderStart, position-1, n);
    root->right = solve (in, pre, index, position+1, inorderEnd, n);
    return root;
}

```



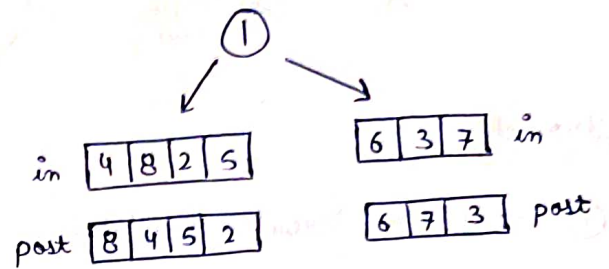
```

Node* buildTree(int in[], int pre[], int n) {
    int preOrderIndex = 0;
    Node* ans = solve(in, pre, preOrderIndex, 0, n-1, n);
    return ans;
}

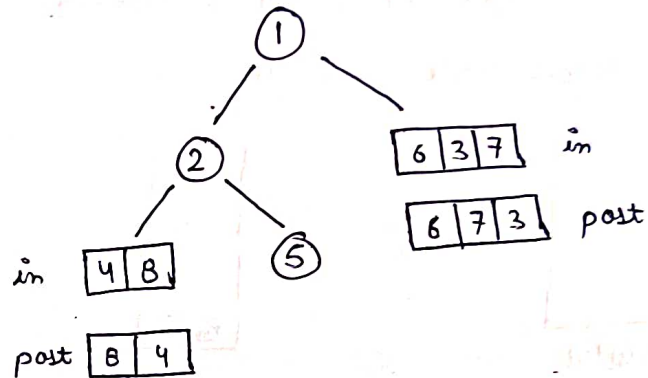
```

Q- Construct Tree from Postorder and Inorder

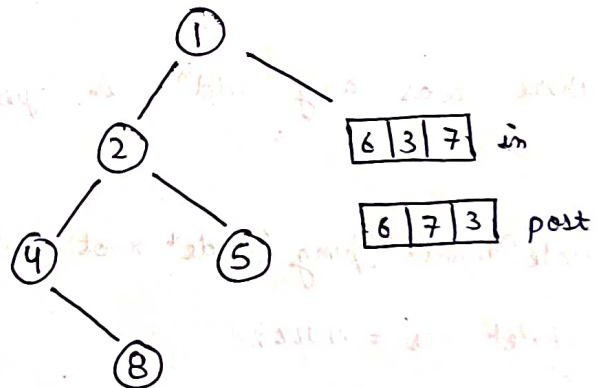
(LNR) in  $\Rightarrow [4 \ 8 \ 2 \ 5 \ 1 \ 6 \ 3 \ 7]$   
 (LRN) post  $\Rightarrow [8 \ 4 \ 5 \ 2 \ 6 \ 7 \ 3 \ 1]$



(LNR) in  $\Rightarrow [4 \ 8 \ 2 \ 5]$   
 (LRN) post  $\Rightarrow [8 \ 4 \ 5 \ 2]$

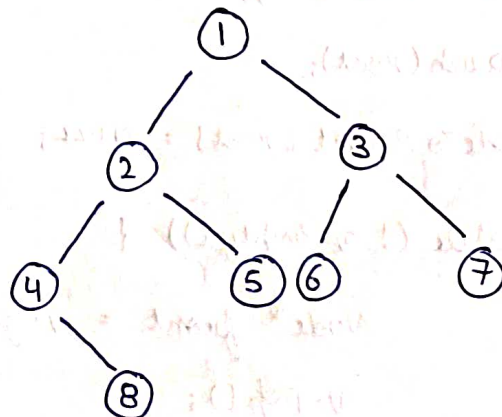


(LNR) in  $\Rightarrow [4 \ 8]$   
 (LRN) post  $\Rightarrow [8 \ 4]$



(LNR) in  $\Rightarrow [6 \ 3 \ 7]$   
 (LRN) post  $\Rightarrow [6 \ 7 \ 3]$

preorder[] = [1 2 4 8 5 3 6 7]  
 (NLR)

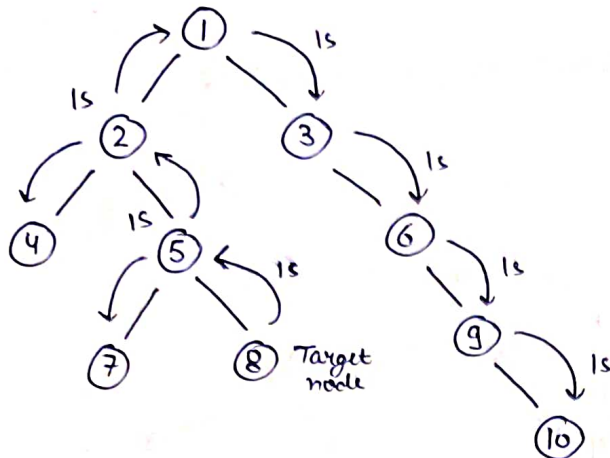


Code is same as previous  
 Bas index me change Hoga  
 & recursive call pehle right  
 wale ki Hogi aur phir left ki



## Qu- Burning Tree

i/p  $\Rightarrow$



o/p  $\Rightarrow$  7

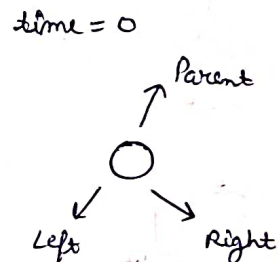
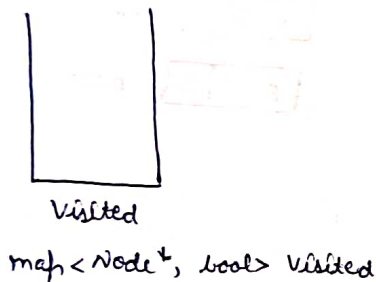
### Approach:-

① node To Parent mapping

map < Node\*, Node\* > nodeToParent (By Level order Traversal)  
O(n)

② Find Target Node O(H)

③



if there was any add<sup>n</sup> in queue (True  $\rightarrow$  time++)  
(False  $\rightarrow$  x)

Node\* createParentMapping (Node\* root, int target, map < Node\*, Node\* > & nodeToParent)

Node\* res = NULL;

queue < Node\* > q;

q.push(root);

nodeToParent[root] = NULL;

while (!q.empty()) {

Node\* front = q.front();

q.pop();

```
if (front -> data == target) {
```

```
    res = front;
```

```
}
```

```
if (front -> left) {
```

```
    nodeToParent[front -> left] = front;
```

```
    q.push(front -> left);
```

```
}
```

```
if (front -> right) {
```

```
    nodeToParent[front -> right] = front;
```

```
    q.push(front -> right);
```

```
}
```

```
}
```

```
return res;
```

```
}
```

```
int burnTree(Node* root, map<Node*, Node*> & nodeToParent) {
```

```
    map<Node*, bool> visited;
```

```
    queue<Node*> q;
```

```
    q.push(root);
```

```
    visited[root] = true;
```

```
    int ans = 0;
```

```
    while(!q.empty()) {
```

```
        bool flag = 0;
```

```
        int size = q.size();
```

```
        for (int i = 0; i < size; i++) {
```

```
            Node* front = q.front();
```

```
            q.pop();
```

T.C. =  $O(n)$

S.C. =  $O(n)$

```
if (front->left && !visited[front->left]) {
```

```
    flag = 1;
```

```
    q.push(front->left);
```

```
    visited[front->left] = 1;
```

```
}
```

```
if (front->right && !visited[front->right]) {
```

```
    flag = 1;
```

```
    q.push(front->right);
```

```
    visited[front->right] = 1;
```

```
}
```

```
if (nodeToParent[front] && !visited[nodeToParent[front]]) {
```

```
    flag = 1;
```

```
    q.push(nodeToParent[front]);
```

```
    visited[nodeToParent[front]] = 1;
```

```
}
```

```
}
```

```
if (flag == 1) {
```

```
    ans++;
```

```
}
```

```
}
```

```
return ans;
```

```
}
```

```
int minTime(Node* root, int target) {
```

```
    map<Node*, Node*> nodeToParent;
```

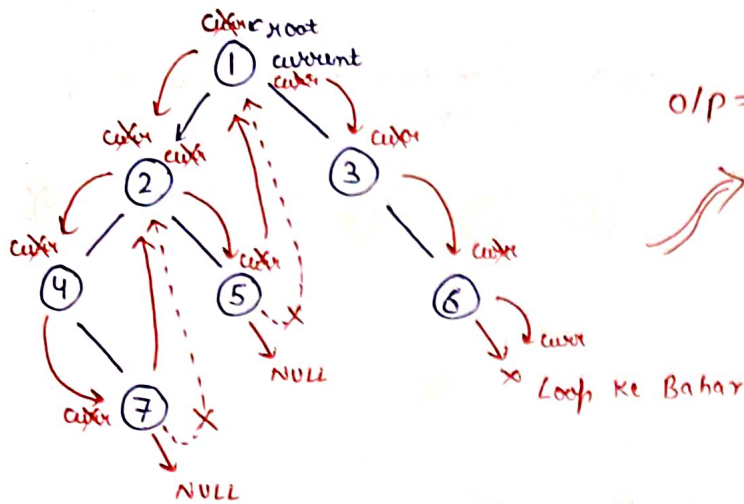
```
    Node* targetNode = createParentMapping(root, target, nodeToParent);
```

```
    int ans = burnTree(targetNode, nodeToParent);
```

```
    return ans;
```

```
}
```

## Q2 - Morris Traversal



### Algo:-

① current  $\rightarrow$  root

② while (~~curr~~  $\neq$  NULL)

if left not exists

visit(curr) // Print

curr = curr  $\rightarrow$  right

else

predecessor = find(curr)

if (pred  $\rightarrow$  right == NULL)

pred  $\rightarrow$  right = curr

curr = curr  $\rightarrow$  left

else

pred  $\rightarrow$  right = NULL

visit(curr)

curr = curr  $\rightarrow$  right

temporary link creation

temporary link removed

### For finding predecessor:-

pred = curr  $\rightarrow$  left

while (pred  $\rightarrow$  right  $\neq$  NULL && pred  $\rightarrow$  right  $\neq$  curr) {

}

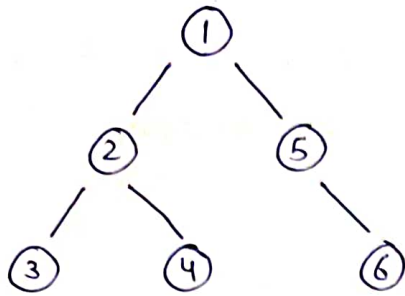
T.C. =  $O(n)$

S.C. =  $O(1)$



Ques- Flatten Binary Tree to Linked List

i/p  $\Rightarrow$



o/p  $\Rightarrow$

Preorder: 1 2 3 4 5 6  
(NLR)



### Approach ①

Traverse karta jaunga aur node create karta jaunga sath sath

But this is wrong because it is not in-place i.e. mujhe given Binary Tree me hi changes karne hai

### Approach ②

Left part L.L. by recursion

Right part L.L. by recursion

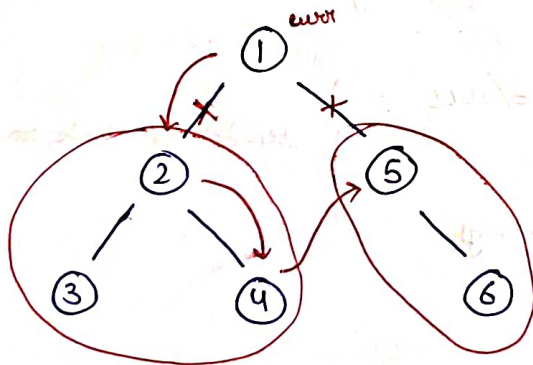
Aur fir connect kar do

T.C. =  $O(n)$

S.C. =  $O(n)$

### Approach ③

Morris Traversal  $\Rightarrow$  S.C. :  $O(1)$



Inorder pred  $\rightarrow$  right = curr  $\rightarrow$  right

$L \rightarrow R$

$N \rightarrow L$

### Algo:-

curr = root

while (curr != NULL)

if curr left exists

pred = curr  $\rightarrow$  left

while (pred  $\rightarrow$  right)

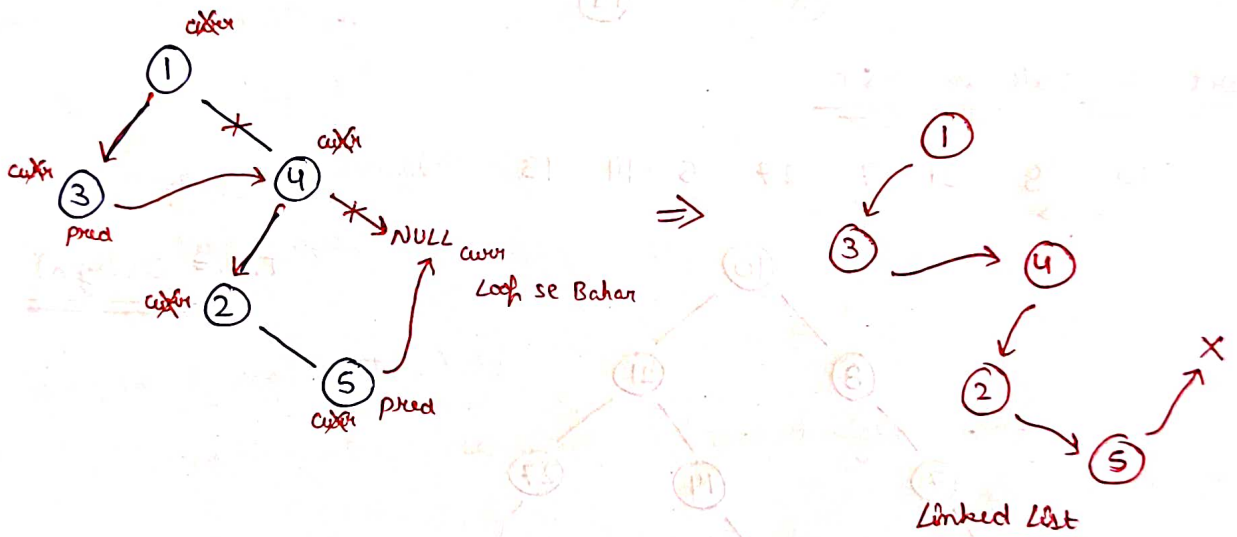
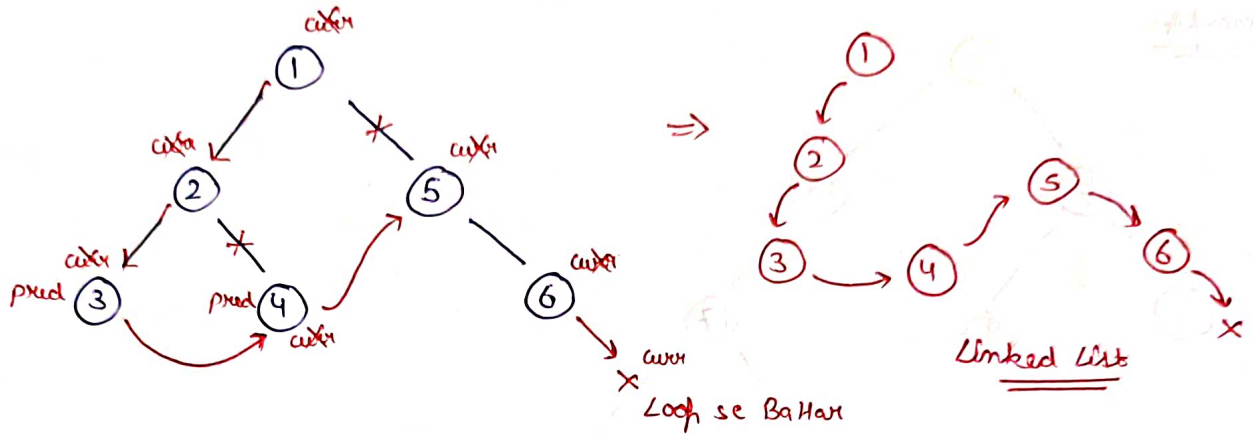
pred → right;

pred → right = curr → right

curr → right = curr → left;

curr = curr → right

Dry Run 1-



Left part ko NULL karne ke liye:-

curr = root;

while (curr != NULL) {

curr → left = NULL;

curr = curr → right;

}